MABELI SENTINEL

Understanding, Code & Defense Handbook

A practical guide for the student team: understand what the system really does, trace the workflow through the source code, use the platform confidently, and defend the project without getting lost in the advanced features.



The one sentence to remember

Mabeli Sentinel is a machine-learning-based attack detection system that receives network-flow data, uses Node.js to coordinate a Python ML prediction, stores the result in MongoDB, and alerts users when suspicious traffic is detected.

How to use this handbook

- First read Sections 1-5. That is enough to understand the project at a high level.
- Then read Sections 6-10 while the source code is open in VS Code. Follow the exact file paths.
- Before presentation day, rehearse the demo sequence in Section 15 and the code-showing sequence in Section 16.
- Use the final cheat sheets when revising with your friend. You do not need to memorize every page or every function.

Important mindset

The project is broad because it contains professional extensions. Your defense does not require you to explain the entire product at once. Defend the core PSA workflow first. Treat multi-organization access, incidents, model evaluation, audit logs, live collectors and other enterprise features as extensions around the same core detection pipeline.

Contents

Section	Topic
1	The project in 60 seconds
2	PSA alignment: what was actually requested
3	Problem, aim, objectives and societal value
4	The architecture you must know
5	Technology roles: Node.js, Python, MongoDB and frontend
6	How to access and navigate the source code
7	Exact end-to-end workflow: click to prediction to alert
8	Core source files: what each one does
9	Real code walkthrough: frontend -> Node.js -> Python -> MongoDB
10	Database records and their relationships
11	Multi-organization design - just enough to defend it
12	What every page means - core vs advanced
13	How to test the system without guessing
14	Common terms in plain English
15	Recommended presentation/demo sequence
16	If they say: Show us the code
17	Hard questions professionals may ask
18	Limitations and what not to claim
19	Study plan for you and your friend
20	Final cheat sheets

1. The project in 60 seconds

> **Say this first**
>
> Mabeli Sentinel is a machine-learning-based web and network attack detection system. It accepts network-flow measurements, sends them through a trained CICIDS2017-based ML model, classifies the traffic as benign or malicious, identifies a likely attack category when malicious, stores the decision in MongoDB and creates alerts for suspicious operational traffic.

The mental model

Network-flow data -> **Express API** -> **ML prediction** -> **Detection record** -> **Alert if suspicious**

Everything else in the application exists around this pipeline. The dashboard summarizes it. Detection History stores it. Alerts prioritize it. Incidents organize serious investigations. Analytics summarizes it. Reports export it. Model Intelligence explains the ML model. Traffic Sources automate the input.

What you do NOT need to think

- You do not need to think of the project as twenty unrelated systems.
- You do not need to memorize every route, MongoDB schema or frontend component.
- You do not need to present every advanced feature during the defense.
- You do need to understand the data path from input -> Node.js -> Python ML -> MongoDB -> result/alert.

2. PSA alignment: what was actually requested

The original PSA direction was straightforward: build a Node.js project with an AI/machine-learning component and MongoDB, using CICIDS2017 to learn the difference between normal and malicious traffic. The current project still follows that goal; the additional professional features sit on top of it.

PSA requirement	How the project satisfies it
Node.js / Express	Main application backend. Receives traffic, calls the ML service, applies risk/alert logic, exposes APIs and serves the frontend.
Python / Machine Learning	Loads the trained model, converts traffic features into the model feature order, predicts benign/malicious, classifies attack type and returns confidence.
MongoDB	Stores organizations, users, traffic events, detections, alerts, incidents, settings, integrations and audit history.
CICIDS2017	Provides labelled examples of benign and malicious network traffic for model training/evaluation.
Dashboard / Web UI	Lets users submit traffic, see detections, investigate alerts, view analytics and manage the platform.

Do not let the extensions change your story

If a lecturer asks what the project is about, answer with the PSA core first. Multi-tenancy, incident management, live ingestion, model evaluation and audit logging are improvements, not a replacement for the original project goal.

3. Problem, aim, objectives and societal value

Problem being solved

Organizations generate large volumes of legitimate network activity alongside potentially malicious activity. Manually examining every network flow is unrealistic, and suspicious behaviour can be buried among normal traffic. The project explores whether machine learning can help identify and prioritize suspicious traffic for human review.

Main aim

Main objective

To design and implement a Node.js-based machine-learning attack detection system that analyzes network traffic, identifies suspicious behaviour, stores the detection result and provides actionable security alerts.

Supporting objectives

- Prepare and evaluate labelled intrusion-detection data.
- Use supervised machine learning to distinguish benign from malicious traffic.
- Classify supported malicious traffic into attack categories.
- Integrate the ML model with a Node.js/Express backend.
- Store traffic events, predictions and alerts in MongoDB.
- Provide a usable dashboard for analysis, alerts, analytics and reports.
- Support future live traffic ingestion through compatible flow collectors.

Value to society and organizations

Value	Meaning
Earlier visibility	Suspicious traffic can be surfaced sooner for investigation.
Reduced manual workload	Analysts can focus on higher-risk traffic rather than checking everything equally.
Education and research	Students can see how ML, backend engineering, databases and cybersecurity connect in one system.
Accessibility	The architecture demonstrates how intelligent monitoring concepts can be brought to smaller organizations, not only large SOCs.
Data-driven reporting	Organizations can turn raw detections into trends, reports and security decisions.

4. The architecture you must know

Core architecture

```
[Browser / User]
       |
       |  traffic features / CSV / live flow
       v
[Node.js + Express API]
       |
       | POST /predict { features }
       v
[Python FastAPI ML Service]
       |
       | prediction + attack_type + confidence
       v
[Node.js Detection Workflow]
       |
       +--> calculate risk/severity
       +--> store TrafficEvent
       +--> store Detection
       +--> create/update Alert if required
       v
[MongoDB]
       |
       v
[Dashboard / Socket.IO real-time update]
```

A simple analogy

Concept	Plain-English analogy
TrafficEvent	The evidence that came into the system.
ML model	The analyst that examines the evidence and gives a classification.
Detection	The written decision produced by the model.
Alert	A suspicious decision important enough to demand attention.
Incident	A formal human investigation created from important alerts.

5. Technology roles: Node.js, Python, MongoDB and frontend

Technology	Responsibility
Frontend (HTML/CSS/JavaScript)	Collects user input, calls backend APIs, renders dashboard data and listens for real-time Socket.IO events.
Node.js + Express	Main coordinator. Handles authentication, organizations, APIs, detection workflow, alert rules, reporting and business logic.
Python + FastAPI	Specialized ML service. Loads the trained model, prepares features and returns predictions.
MongoDB + Mongoose	Persistent storage. Keeps users, organizations, traffic, detections, alerts, incidents and configuration.
Socket.IO	Pushes new detections/alerts to the correct organization workspace without requiring a page refresh.
Docker Compose	Starts the Node.js app, Python ML service and MongoDB together for local deployment.

> **Best short explanation**
>
> Node.js is the application brain, Python is the machine-learning engine, MongoDB is the operational memory, and the browser is the user interface.

6. How to access and navigate the source code

Open the project in VS Code

1. Extract the project ZIP to a normal folder on your computer.

2. Open VS Code.

3. Choose File -> Open Folder and select the mabeli-sentinel folder.

4. Or open a terminal in the folder and run: code .

Windows / VS Code

```
cd path\to\mabeli-sentinel
code .
```

The folder tree you should recognize

Current project structure

```
mabeli-sentinel/
```

```
|-- frontend/
|   `-- public/
|       |-- index.html
|       |-- app.js
|       |-- styles.css
|       `-- test-data/
|
|-- backend/
|   `-- src/
|       |-- server.js
|       |-- routes/api.js
|       |-- services/detection.js
|       |-- services/ml.js
|       |-- services/tenant-context.js
|       |-- models/index.js
|       `-- middleware/auth.js
|
|-- ml-service/
|   |-- app/main.py
|   |-- models/
|   `-- data/
|
|-- collector/
|   `-- flow_forwarder.py
|
`-- docker-compose.yml
```

The eight files to learn first

File	Why you care
frontend/public/app.js	Shows how the browser calls the backend and displays results.
backend/src/server.js	Starts Express, connects MongoDB, serves the frontend and configures Socket.IO.
backend/src/routes/api.js	Contains the API endpoints such as /analysis/single and /analysis/batch.
backend/src/services/detection.js	The single most important application workflow: predict -> risk -> detection -> alert.
backend/src/services/ml.js	The small bridge from Node.js to the Python ML API.
backend/src/models/index.js	Defines the MongoDB records and how they relate.
ml-service/app/main.py	Contains the ML prediction and training logic.
docker-compose.yml	Shows how the application services are started together.

> **What NOT to do during a defense**
>
> Do not randomly scroll through the repository. Open these files in a planned order and explain one continuous workflow.

7. Exact end-to-end workflow: click to prediction to alert

Example: the user clicks Run model analysis

Step	What happens
1	The frontend reads the JSON network-flow features entered or loaded by the user.
2	frontend/public/app.js sends POST /api/analysis/single with {features, meta}.
3	backend/src/routes/api.js validates the body and calls processTraffic(...).
4	processTraffic() creates a TrafficEvent record so the incoming evidence is preserved.
5	Node.js calls predict(features) in backend/src/services/ml.js.
6	ml.js sends POST /predict to the Python FastAPI service.
7	Python reorders/fills the feature values to match the trained model and runs binary prediction.
8	If malicious, Python also chooses the most likely attack class and returns confidence/probabilities.
9	Node.js calculates a risk score and severity and creates a Detection record.
10	If the prediction is malicious and crosses the organization threshold, Node.js creates or updates an Alert.
11	Socket.IO emits the new detection only to browsers inside that organization workspace.
12	The frontend displays the result: attack type, benign/malicious, confidence, risk and alert status.

8. Core source files: what each one does

8.1 frontend/public/app.js

This is the main browser-side JavaScript. It manages login state, organization selection, API calls, page rendering and actions such as single analysis, batch upload, alert updates and user management.

Important idea: every request carries login + organization context

```
async function api(path, options={}) {
  const headers={...(options.headers||{})};
  if(token) headers.Authorization=`Bearer ${token}`;
  if(token && currentOrganization?._id)
    headers['X-Organization-Id']=currentOrganization._id;
```

```
  ...
  return fetch(`${API}${path}`, {...options, headers});
}
```

> **What to say**
>
> The frontend never talks directly to MongoDB or the ML model. It communicates with the Node.js API.

8.2 backend/src/server.js

This file boots the Node.js application. It checks environment variables, creates Express and Socket.IO, applies security middleware, mounts the /api router, serves the frontend, connects to MongoDB and starts listening on the configured port.

- Express receives normal HTTP API calls.
- Helmet/CORS/rate limiting provide baseline web protections.
- Socket.IO provides real-time updates.
- Membership is checked before a socket joins an organization room.
- MongoDB is connected before the server begins listening.

8.3 backend/src/routes/api.js

This is the main API router. For your defense, the most useful route is POST /analysis/single because it clearly demonstrates the PSA workflow.

The route validates input and hands the real work to processTraffic()

```
router.post('/analysis/single', requireAuth, asyncHandler(async (req, res) => {
  const input = ...parse(req.body);
  const result = await processTraffic({
    features: input.features,
    meta: {...},
    io: req.app.get('io')
  });
  res.status(201).json(result);
}));
```

8.4 backend/src/services/detection.js

> **Most important file**
>
> If they ask you to show the heart of the Node.js project, open this file and find export async function processTraffic(...).

processTraffic() is the business workflow that turns one incoming flow into a stored security decision. It creates the traffic record, asks the ML service for a prediction, calculates risk/severity, stores the detection, optionally creates/correlates an alert, and emits a real-time event.

8.5 backend/src/services/ml.js

This file deliberately stays small. It uses Axios to call the Python ML service. That separation is useful because Node.js handles the application while Python handles machine learning.

Node.js -> Python bridge

```
export async function predict(features) {
  const { data } = await client.post('/predict', { features });
  return data;
}
```

8.6 ml-service/app/main.py

This is the core Python ML service. It exposes FastAPI endpoints such as /health, /model, /predict, /predict/batch and /train. For defense, focus on run_prediction(features).

- vectorize() aligns the incoming feature names with the trained model feature order.
- The binary model estimates benign vs malicious probability.
- If malicious, the multiclass model estimates the most likely attack class.
- The service returns confidence, malicious probability, per-class probabilities, explanation features and model metadata.

8.7 backend/src/models/index.js

This file defines the MongoDB/Mongoose schemas. You do not need to memorize all fields. Learn the important entities and how they connect.

8.8 backend/src/middleware/auth.js and tenant-context.js

These files support the multi-organization extension. Authentication confirms the user, selects one active organization membership and runs the request inside that organization context. Tenant-aware models then automatically scope organization-owned records.

> **This is an extension, not the PSA core**
>
> Only go deep into tenancy if somebody asks how multiple organizations can safely use the platform. It is not the first code you should show.

9. Real code walkthrough: frontend -> Node.js -> Python -> MongoDB

9.1 Frontend sends a single analysis

From frontend/public/app.js

```
const r = await api('/analysis/single', {
  method: 'POST',
  body: JSON.stringify({
    features,
    meta: { environment, sourceName }
  })
});
```

Meaning: the browser sends one JavaScript object containing network-flow measurements to the Node.js backend.

9.2 Express receives and validates it

From backend/src/routes/api.js

```
router.post('/analysis/single', requireAuth, asyncHandler(async (req, res) => {
  const input = z.object({
    features: z.record(z.any()),
    meta: z.record(z.any()).optional()
  }).parse(req.body);
  const result = await processTraffic({ features: input.features, ... });
  res.status(201).json(result);
}));
```

Meaning: authentication is required, the shape of the request is checked, and then the backend calls the central detection workflow.

9.3 Node.js preserves the incoming traffic

From processTraffic() in detection.js

```
const traffic = await TrafficEvent.create({
  organization: organizationId,
  source: ...,
  environment,
  sourceName: ...,
  features
});
```

Meaning: the system records what it received before turning the ML result into a detection. This provides traceability.

9.4 Node.js asks Python to predict

detection.js

```
const result = await predict(features);
```

ml.js

```
export async function predict(features) {
  const { data } = await client.post('/predict', { features });
  return data;
}
```

Meaning: Node.js remains the main application backend but delegates the specialist ML calculation to Python over an internal HTTP API.

9.5 Python prepares and classifies the record

From run_prediction() in ml-service/app/main.py

```
row = vectorize(features)
binary_probs = binary.predict_proba(row)[0]
malicious_probability = bmap.get('malicious', 0.0)
prediction = 'malicious' if malicious_probability >= 0.5 else 'benign'
```

Meaning: the binary classifier produces a probability. The current implementation treats 0.5 or more malicious probability as malicious at the ML classification stage.

9.6 If malicious, Python chooses an attack class

Multiclass classification

```
multi_probs = multi.predict_proba(row)[0]
malicious_probs = { ... }
attack_type = max(malicious_probs, key=malicious_probs.get)
confidence = malicious_probs.get(attack_type, malicious_probability)
```

Meaning: the system separates the question "Is it malicious?" from "What type of malicious traffic does it most resemble?"

9.7 Node.js converts the prediction into operational security data

Stored detection

```
const riskScore = scoreRisk(prediction, attackType, confidence);
const severity = severityFor(riskScore, settings.severityThresholds);

const detection = await Detection.create({
  trafficEvent: traffic._id,
  prediction,
  attackType,
  confidence,
  riskScore,
  severity,
  modelVersion: result.model_version
});
```

Meaning: an ML answer becomes a structured security record that the dashboard can search, filter, report and investigate.

9.8 Suspicious operational traffic may create an alert

Alert threshold and correlation

```
if (prediction === 'malicious' &&
    maliciousProbability >= settings.maliciousThreshold) {
  // create a new alert or group it with a recent related alert
}
```

Meaning: not every stored record becomes a human-facing alert. The application uses the organization threshold and grouping logic to reduce unnecessary alert noise.

9.9 Real-time update

Organization-scoped Socket.IO

```
io?.to(`org:${organizationId}`).emit('detection:new', { detection, traffic });
```

Meaning: only users connected to the same organization workspace receive that live detection event.

10. Database records and their relationships

Simplified data model

```
Organization
  |
  +-- Membership -- User
  |
  +-- TrafficEvent --> Detection --> Alert --> Incident
  |
  +-- Integration
  +-- Setting
  +-- AuditLog
```

Record	Meaning
Organization	Customer/workspace using Mabeli Sentinel.
User	A human account.
Membership	Connects a user to an organization with owner/admin/analyst/viewer permissions.
TrafficEvent	The network-flow data submitted to the system.
Detection	The ML decision for a traffic event.
Alert	A suspicious detection/group requiring analyst attention.
Incident	A formal investigation that may link one or more alerts.
Integration	A trusted live traffic source/API key.
Setting	Organization-specific alert and severity thresholds.
AuditLog	Records important actions for accountability.

> **Core relationship to memorize**
>
> TrafficEvent -> Detection -> Alert -> Incident. The first two are automatic data/ML records. The last two are human security workflow records.

11. Multi-organization design - just enough to defend it

The multi-organization capability was added so Mabeli Sentinel can behave like a platform offered by Mabeli Technology rather than a one-company hard-coded application. It is useful, but it is not the main PSA research question.

What happens when a user logs in

JWT user	->	Membership	->	Organization	->	Tenant context	->	Scoped data

- A user can belong to one or more organizations.

- Membership stores the role inside each organization: owner, admin, analyst or viewer.
- The frontend includes X-Organization-Id on API calls.
- requireAuth verifies the token and confirms that the user has an active membership in the requested organization.
- Tenant-aware data models automatically scope records to the current organization.
- Socket.IO also checks membership and joins org:<organizationId>, preventing cross-organization live events.

If they ask why this was added

> **Suggested answer**
>
> The PSA system was originally organization-focused, but a hosted platform should not let one company see another company's security data. Multi-tenancy allows multiple organizations to use the same application while isolating their users, traffic, detections, alerts and settings.

If they ask whether it changed the project goal

> **Suggested answer**
>
> No. The detection pipeline is unchanged: traffic -> Node.js -> ML -> MongoDB -> alerts. Multi-tenancy changes who owns each record, not how attack detection works.

12. What every page means - core vs advanced

When you first enter the platform, the number of pages can feel overwhelming. Use this table to separate the PSA core from professional extensions.

Page	Level	What it is
Getting Started	Core helper	Guides a new organization through first analysis, team setup and live monitoring.
Command Overview	Core	Summary of analyzed traffic, detections, alerts and current security posture.
Single Analysis	Core	Submit one compatible network-flow record to the ML model.
Batch Analysis	Core	Upload a CSV containing many network-flow records.
Detection History	Core	The stored results produced by the ML model.
Alerts	Core	Suspicious operational detections above the configured threshold.
Reports	Core	Export security activity for technical or management review.
Live Operations	Advanced operational	Shows incoming operational detections in real time.
Incidents	Advanced	Formal human investigations built from alerts.

Page	Level	What it is
Analytics	Advanced	Aggregates stored detections into trends and distributions.
Model Intelligence	Advanced / research	Shows active model, metrics, classes, features and retraining.
Traffic Sources	Advanced / deployment	Connects a flow collector/API key for live ingestion.
Team & Access	Advanced / SaaS	Manages organization owners, admins, analysts and viewers.
Audit Log	Advanced / governance	Records who changed what and when.
Detection Settings	Advanced / tuning	Controls alert threshold, grouping and severity bands.
Test Centre	Advanced / verification	Exercises workflows using isolated test traffic.
Help & How It Works	Core helper	Explains terms, data formats and system use.

> **Defense advice**
>
> Do not click every page. Demonstrate one traffic analysis, one stored detection, one alert, the model information and the code flow. Mention the rest as extensions.

13. How to test the system without guessing

Built-in files

The project includes reusable traffic fixtures under frontend/public/test-data/. They are there so you and your friend can understand the workflow without inventing network values.

Test data

```
frontend/public/test-data/
|-- single-benign.json
|-- single-portscan.json
|-- batch-labelled.csv
`-- batch-unlabelled.csv
```

File	Use
single-benign.json	One genuine CICIDS2017-derived benign flow. Use for Single Analysis.
single-portscan.json	One genuine CICIDS2017-derived PortScan-labelled flow. Use for Single Analysis.

File	Use
batch-labelled.csv	Multiple labelled records. Good for model evaluation/benchmarking.
batch-unlabelled.csv	Traffic without ground-truth labels. Good for normal operational/test workflow.

Recommended learning test

1. Start the project and sign in.
2. Open Single Analysis.
3. Load the benign fixture and run analysis. Observe prediction, confidence, risk and whether an alert was created.
4. Repeat with the PortScan fixture.
5. Open Detection History and find the two stored decisions.
6. Open one detection and inspect its traffic features, explanation, model version and risk score.
7. If a malicious operational/test record crossed the threshold, open Alerts and inspect the linked alert.
8. Open Model Intelligence and note the active algorithm, version, classes and metrics.
9. Open the source code and trace that same action through app.js -> api.js -> detection.js -> ml.js -> main.py -> models/index.js.

> **The learning trick**
>
> Do not learn the code separately from the running app. Perform one action, then trace exactly which files handled it. That is the fastest way to stop the project feeling abstract.

14. Common terms in plain English

Term	Plain-English meaning
Network flow	A summarized conversation between network endpoints, represented by measurements such as duration, packet counts and byte sizes.
Feature	One measurable input used by the model, for example Flow Duration or Destination Port.
Benign	Traffic the model considers normal/non-malicious.
Malicious	Traffic the model considers suspicious or attack-like.
Prediction	The model's classification result.
Confidence	How strongly the model supports the selected class. It is not a guarantee of correctness.
Malicious probability	The binary model's estimated probability that the flow is malicious.
Attack type	The malicious class the multiclass model considers most likely.
Risk score	Application-level score combining attack type and confidence for prioritization.

Term	Plain-English meaning
Severity	Low, medium, high or critical band derived from the risk score.
Detection	One persisted ML decision.
Alert	A suspicious detection/group that crossed the alert threshold.
Incident	A human-managed investigation created from important alerts.
False positive	Benign traffic incorrectly classified as malicious.
False negative	Malicious traffic incorrectly classified as benign.
Precision	When the model says attack, how often is it correct?
Recall	Of the actual attacks, how many did the model detect?
F1 score	A balance between precision and recall.
Benchmark	Labelled traffic used to measure model performance.
Tenant / organization	One isolated customer workspace in the multi-organization platform.

15. Recommended presentation/demo sequence

A clean 8-10 minute technical demo

Stage	What to do/say
1. Problem (45 sec)	Organizations cannot manually inspect every network flow. We use ML to help identify suspicious traffic.
2. Architecture (45 sec)	Show Traffic -> Node.js -> Python ML -> MongoDB -> Alert/Dashboard.
3. System ready (30 sec)	Show that backend, database and ML service are online.
4. Run one analysis (90 sec)	Use a provided benign or PortScan fixture. Explain what data is being submitted.
5. Show detection (60 sec)	Open the stored decision: prediction, class, confidence, risk, explanation and model version.
6. Show alert logic (45 sec)	Explain that only suspicious operational/test traffic above threshold creates/correlates alerts.
7. Show model page (45 sec)	Point out the active model, dataset/training scope and evaluation metrics.
8. Show code (2-3 min)	Trace app.js -> api.js -> detection.js -> ml.js -> main.py -> models/index.js.
9. Limitations (45 sec)	Acknowledge flow-level features, CICIDS2017 age, false positives/negatives and need for live feature extraction.
10. Close (30 sec)	Restate value: earlier visibility, prioritization and a practical bridge between ML research and an operational application.

> **Do not start with the sidebar**
>
> If you begin by clicking through advanced pages before explaining the pipeline, the audience will experience the same confusion your friend experienced. Establish the mental model first.

16. If they say: "Show us the code"

Use this exact order

Order	File	What to say
1	backend/src/services/detection.js	Open processTraffic(). Say: "This is the central Node.js detection workflow."
2	backend/src/services/ml.js	Show predict(features). Say: "Node.js sends the traffic features to the Python service here."
3	ml-service/app/main.py	Show run_prediction(). Explain binary prediction and attack classification.
4	backend/src/models/index.js	Show TrafficEvent, Detection and Alert schemas. Explain persistence.
5	backend/src/routes/api.js	Show /analysis/single. Explain how the HTTP request reaches processTraffic().
6	frontend/public/app.js	Show the call to /analysis/single if they want the browser side.
7	backend/src/server.js	Only if asked how the application starts or how real-time organization rooms work.

A 90-second code explanation

> **Script**
>
> "The browser sends traffic features to our Express endpoint. The endpoint calls processTraffic(), which stores the incoming traffic and sends the feature object to the Python ML service through ml.js. Python aligns the features with the trained model, predicts benign or malicious, and if malicious selects an attack class. The prediction returns to Node.js, where we calculate risk and severity, store a Detection in MongoDB, and create or correlate an Alert if the configured threshold is exceeded. Socket.IO then pushes the result to the organization dashboard."

What not to open first

- Do not begin with tenant-context.js unless they ask about multi-organization isolation.
- Do not begin with CSS or the giant frontend renderer; it hides the core idea.
- Do not begin with Docker Compose unless they ask about deployment.
- Do not spend five minutes explaining authentication before showing the detection pipeline.

17. Hard questions professionals may ask

Question	Defensible answer
Why machine learning instead of only rules/signatures?	Rule/signature systems remain valuable. ML is used as a complementary statistical detection layer that learns patterns in labelled traffic rather than relying only on manually written signatures.
Why CICIDS2017 when it is old?	It is a reproducible labelled academic intrusion-detection baseline with benign and multiple attack categories. Its age is acknowledged as a limitation; production deployment should evaluate newer and organization-specific traffic.
Does the system detect every attack?	No. It detects patterns represented by the model and features. It should be one layer in a broader defense-in-depth architecture.
Does it stop attacks?	No. The current system detects, records, prioritizes and supports investigation. Automated blocking would be a separate prevention capability.
Can it detect zero-days?	Not reliably by definition. A supervised classifier may flag unusual traffic that resembles known malicious patterns, but universal zero-day detection cannot be claimed.
How does live traffic reach the model?	A compatible network-flow collector/feature extractor must convert raw traffic into the same feature schema expected by the trained model, then send those features to the ingestion API.
Can it analyze a website URL directly?	No. The model expects network-flow measurements, not simply a URL or page content.
What about HTTPS?	Flow metadata such as timing, packet counts and sizes can still exist, but encrypted payloads reduce application-content visibility.
Why Node.js and Python together?	Node.js is the main application/backend and Python provides mature ML tooling. A small HTTP service boundary keeps both responsibilities clear.
Why MongoDB?	The application stores flexible document-like security records such as feature maps, model explanations, alert notes and timelines. PostgreSQL would also be valid; MongoDB was chosen for development fit and flexible documents.
How do you avoid alert fatigue?	Predictions are stored as detections, but alerts are created only when malicious probability passes an organization threshold. Related recent alerts can also be grouped by attack/source/destination.
What happens when the model is wrong?	Analysts can review outcomes and mark false positives. Model metrics, benchmark evaluation and future retraining are used to monitor/improve performance.
What is model drift?	Real-world traffic changes over time, so the training distribution may stop representing current conditions. Production use requires ongoing evaluation and retraining with appropriate data.
Can attackers fool the model?	Potentially. Evasion/adversarial behaviour is a real limitation. The model should complement other security controls, not replace them.
Why have multi-organization support?	A hosted platform must isolate each customer's security data. Multi-tenancy adds ownership/isolation without changing the core detection pipeline.

Question	Defensible answer
How is cross-company data leakage prevented?	API requests are tied to an active organization membership; tenant-aware models scope organization-owned data; live Socket.IO connections also join only that organization's room.
Is this production-ready?	It is a strong academic/product prototype. Production deployment would require broader current datasets, load testing, infrastructure hardening, monitoring, backup/recovery, compliance and continuous model evaluation.
What is the main contribution?	The project integrates a trained network-traffic classifier into a complete Node.js workflow: ingestion, persistence, risk scoring, alerts, investigation, analytics and reporting.

18. Limitations and what not to claim

Limitations you should mention yourself

- CICIDS2017 is useful for reproducible research but does not perfectly represent all modern network environments or attacks.
- The model depends on network-flow features. Live deployment needs compatible feature extraction before prediction.
- ML predictions can produce false positives and false negatives.
- Confidence is model certainty, not proof that a prediction is correct.
- The current risk score includes application-defined attack weights; it is a prioritization aid, not a universal industry risk standard.
- Encrypted traffic limits payload-level visibility.
- A supervised model cannot guarantee detection of unseen attack techniques.
- The application is detection/decision support; it does not automatically block attacks by default.

Never say these things

Do not claim	Better wording
"It detects every attack."	"It detects attack patterns supported by its training data and feature representation."
"It stops hackers."	"It detects and prioritizes suspicious traffic for investigation."
"99% accuracy means 99% secure."	"We evaluate accuracy together with precision, recall, F1 and false-positive/false-negative behaviour."
"It detects zero-days."	"It may flag unfamiliar traffic, but reliable zero-day detection is not guaranteed."
"The CICIDS benchmark attacks happened to Mabeli."	"Benchmark data is labelled research traffic used for evaluation, not evidence of attacks against an organization."
"AI replaces the analyst."	"AI reduces manual triage and supports human decision-making."

19. Study plan for you and your friend

Session 1 - 45 minutes: understand the story

- Memorize the one-sentence project definition.
- Draw the five-box architecture from memory.
- Explain TrafficEvent -> Detection -> Alert -> Incident to each other.
- Explain what Node.js, Python and MongoDB each do without looking at notes.

Session 2 - 60 minutes: run the system

- Run one benign single analysis and one malicious-labelled test fixture.
- Find the resulting detections in Detection History.
- Open a detection and identify prediction, attack type, confidence, risk, model version and explanation.
- Open Alerts and understand why an alert did or did not appear.

Session 3 - 75 minutes: trace the code

- Open app.js and locate /analysis/single.
- Open api.js and locate router.post(/analysis/single).
- Open detection.js and locate processTraffic().
- Open ml.js and locate predict(features).
- Open main.py and locate run_prediction(features).
- Open models/index.js and locate TrafficEvent, Detection and Alert.
- One person explains while the other interrupts with "why?" questions.

Session 4 - 60 minutes: mock defense

- One person acts as lecturer/professional and asks 10 questions from Section 17.
- Swap roles.
- Practice saying limitations without sounding apologetic.
- Practice the 90-second source-code walkthrough twice without notes.

The rule for both of you

> **If you cannot explain a feature simply, do not make it the centre of the presentation**
>
> You can truthfully say: "That is an advanced extension around the core detection pipeline. The part central to our PSA is..." and return to the workflow you understand.

20. Final cheat sheets

20.1 One-page mental model

Revision sheet

```
WHAT THE PROJECT DOES
---------------------
Input:     network-flow measurements
Backend:   Node.js / Express
AI:        Python FastAPI + trained classifier
```

```
Storage:   MongoDB
Output:    benign/malicious, attack class, confidence, risk
Action:    store Detection; create Alert if suspicious enough

THE CORE FLOW
-------------
Browser -> /api/analysis/single -> processTraffic()
        -> ml.predict(features) -> Python /predict
        -> run_prediction() -> result
        -> risk/severity -> Detection -> optional Alert
        -> MongoDB + Socket.IO -> dashboard

THE 6 FILES TO SHOW
-------------------
1. backend/src/services/detection.js
2. backend/src/services/ml.js
3. ml-service/app/main.py
4. backend/src/models/index.js
5. backend/src/routes/api.js
6. frontend/public/app.js
```

20.2 What to say when asked "What did you build?"

> **30-second answer**
>
> "We built Mabeli Sentinel, a Node.js-based network attack detection platform. Users provide network-flow data manually, by CSV or through a compatible collector. Express sends those features to a Python machine-learning service trained around CICIDS2017-style traffic. The model returns a benign or malicious classification, attack class and confidence. Node.js calculates risk, stores the traffic and detection in MongoDB, and creates alerts for suspicious traffic above the configured threshold. The rest of the platform provides investigation, analytics, reports and multi-organization access around that core pipeline."

20.3 What to say when asked "Where is the AI?"

> **Answer**
>
> "The AI is implemented in ml-service/app/main.py. The run_prediction() function receives the flow features, vectorizes them into the model feature order, uses a binary classifier for benign versus malicious and a multiclass classifier for attack type, then returns confidence, probabilities and explanation information to Node.js."

20.4 What to say when asked "Where is Node.js?"

> **Answer**
>
> "Node.js is the main application backend under backend/src. The central workflow is processTraffic() in services/detection.js. It coordinates persistence, the call to the ML service, risk/severity calculation, alert creation and real-time updates."

20.5 What to say when asked "Where is MongoDB?"

> **Answer**
>
> "MongoDB stores the operational records. The Mongoose schemas are defined in backend/src/models/index.js. The most important relationship is TrafficEvent -> Detection -> Alert -> Incident."

20.6 What to say when you genuinely do not know

> **Professional response pattern**
>
> Do not invent an answer. Say: "I do not want to guess. In our current implementation, the relevant part works this way..." Explain what is implemented, state the limitation, and describe what would be required to extend it. That is stronger than pretending the system does something it does not.

Closing reminder

The project feels complex only when every feature is treated as equally important. It is not. The core project is one continuous pipeline: receive traffic -> call the ML model -> store the decision -> alert on suspicious activity. Learn that pipeline deeply, know the six core files, understand the basic data records, and use the advanced features only when the discussion reaches them.

> **The sentence to leave in your head**
>
> Traffic -> Node.js -> Python ML -> Prediction -> MongoDB -> Alert / Dashboard.

Appendix A. Quick local startup reference

Start all services

```
docker compose up --build
```

Then open http://localhost:4000 in your browser. To stop the project:

Stop services

```
docker compose down
```

> **Be careful with volumes**
>
> docker compose down -v removes the MongoDB volume and deletes stored users/security records. Do not use -v unless you intentionally want a clean database.

Appendix B. Fast source-code index

Area	Where to look
Login / organizations	frontend/public/app.js; backend/src/routes/api.js; backend/src/middleware/auth.js
Single analysis	frontend/public/app.js -> POST /analysis/single -> backend/src/routes/api.js
Core detection workflow	backend/src/services/detection.js -> processTraffic()
Node -> Python bridge	backend/src/services/ml.js -> predict(features)
ML inference	ml-service/app/main.py -> run_prediction(features)
MongoDB schemas	backend/src/models/index.js
Real-time updates	backend/src/server.js + detection.js Socket.IO emit
Multi-organization scoping	middleware/auth.js + services/tenant-context.js + tenantPlugin
Batch analysis	backend/src/routes/api.js -> POST /analysis/batch
Training/model endpoints	ml-service/app/main.py -> /model, /train, /predict
Live collector	collector/flow_forwarder.py + integrations/ingest routes in api.js
Deployment	docker-compose.yml + backend/Dockerfile + ml-service/Dockerfile

Prepared for the Mabeli Sentinel PSA project team.